

第4章：神经网络学习

张正



哈尔滨工业大学（深圳）

zhengzhang@hit.edu.cn

<http://faculty.hitsz.edu.cn/zhangzheng>

本章内容

一、神经网络的定义

二、神经网络的发展历史

三、感知器模型

四、梯度下降法则和Delta法则

五、多层神经网络和反向传播算法

六、深层神经网络

七、人脸识别的应用

一、神经网络的定义

神经网络的命名：

- ✓ 神经网络（Neural Networks）
- ✓ 人工神经网络（Artificial Neural Networks）
- ✓ 人工神经系统（Artificial Neural Systems）
- ✓ 神经计算机（Neural Computers）
- ✓ 自适应系统（Adaptive Systems）
- ✓ 自适应网（Adaptive Networks）
- ✓ 联结主义（Connectionisms）

一、神经网络的定义

“神经网络是由具有适应性的简单单元组成的广泛并行互联的网络，它的组织能够模拟生物神经系统对真实世界物体所作出的反应。” [Kohonen, 1988]

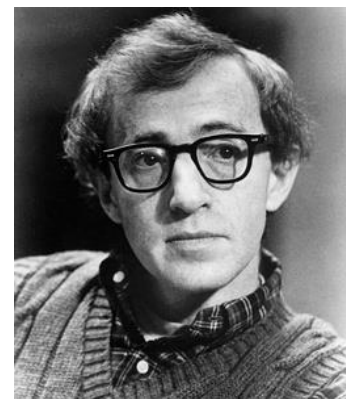
- 机器学习中的神经网络通常是指“神经网络学习”，或者说是机器学习与神经网络两个学科的交叉部分。
- 神经网络提供了一种逼近实值、离散值和向量值函数的健壮方法
 - 对于某些类型的问题，例如学习解释复杂的现实世界的传感器数据，神经网络是非常有效的
- 神经网络的研究是由试图模拟生物神经系统受启发而来的。因此，有必要先来看看生物神经系统的工作过程：

一、神经网络的定义

人脑

人是地球上具有最高智慧的动物, 而人的智慧均来自大脑, 人类靠大脑进行思考、联想、记忆和推理判断等, 这些功能是什么任何被称为“电脑”的一般计算机所无法取代的。

“My brain: It’s my second favorite organ.”



伍迪·艾伦 (Woody Allen)

一、神经网络的定义

人脑

每个神经元可看作是一个小的处理单元，这些神经元按某种方式连接起来，形成大脑内部的生理神经元网络。

神经元网络中各神经元之间联结的强弱，按外部的激励信号做自适应变化，而每个神经元又随着所接收到的多个接收信号的综合大小而呈现兴奋或抑制状态。

现已明确大脑的学习过程就是神经元之间连接强度随外部激励信息做自适应变化的过程，而大脑处理信息的结果则由神经元的状态表现出来。

一、神经网络的定义

● 生物神经系统

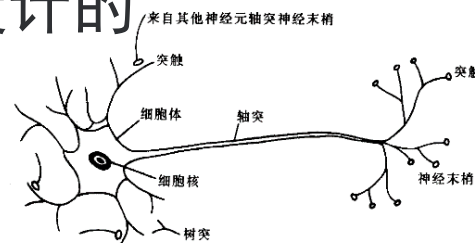


- 神经元之间通过**突触**两两相连。
- **轴突**记录了神经元间联系的强弱。
- 只有达到一定的兴奋程度，神经元才向外界传输信息。
- 每个神经元可抽象成一个激励函数（非线性处理）。

让计算机更加鲁棒、更加智能、且具有学习功能!

一、神经网络的定义

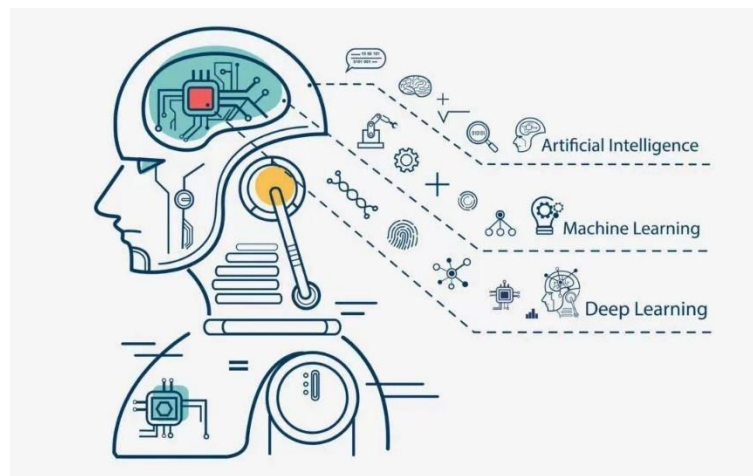
- 生物学习系统是由相互连接的神经元组成的非常复杂的网络构成的。
 - 人类的大脑是一个紧密相连的网络 10^{11} (神经元, 每一个链接其他 10^4)
 - 神经元的活动通常通过与其他神经元的连接而被激活或抑制。
- 生物神经系统的信息处理能力
 - 与计算机速度相比, 神经元的切换时间较慢, 然而人类能够快速做出复杂的决定
 - 基于分布式表示的高度并行计算
- 人工神经网络系统就是为了捕捉这种能力而设计的



一、神经网络的定义

- 人工神经网络的两个流派

- 利用神经网络来研究和模拟生物学习过程
- 获得高效的机器学习算法，而不依赖于这些算法是否反映了生物过程（该课程的关注点）



二、神经网络的发展历史

可以说，神经网络的发展是非常曲折的，从诞生到现在，几经兴衰。大体上，可以将其发展历史分成如下五个时期：

- ✓ 萌芽期（人类研究自己智能的开始~1949）
- ✓ 第一次高潮期（1950~1968）
 - 反思期（1969~1982）
- ✓ 第二次高潮期（1983~1990）
 - 再认识与应用期（1991~2012）
 - 第三次浪潮（2012~）

二、神经网络的发展历史

萌芽期（人类研究自己智能的开始~1949）：

- 人类对神经网络的研究最早可以追溯到人类研究自己智能的开始。
- 1943年，心理学家McCulloch和数学家Pitts建立起了著名的阈值加权和模型，简称为M-P模型，并从原理上证明了人工神经网络能够计算任何算数和逻辑函数。
- 1949年，心理学家Hebb提出神经元之间的突触联系是可变的假说——Hebb学习规律。Hebb学习规律被认为是神经网络学习算法的里程碑。
- 这两个重大的研究成果，构成了神经网络萌芽期的标志。

二、神经网络的发展历史

第一次高潮期（1950~1968）：

- 1958年，Rosenblatt提出了**单层感知机**(Perceptron)**模型**及其学习规则。
- 单层感知机的成功标示着神经网络研究的**第一高潮期的到来**。
- 第一高潮期的成功让人们乐观地**认为几乎已经找到了智能研究**的关键。因此，许多部门都开始大规模地投入此项研究，希望尽快占领神经网络研究的制高点。

二、神经网络的发展历史

反思期（1969~1982）：

- 1969年，正当人们兴奋不已的时候，Minsky和Papert发表了《Perceptrons》一书，明确指出：单层感知机不能解决非线性问题，多层网络的训练算法尚无希望。这一论断导致神经网络跌入谷底。
- 这一成果的发表标志着人类对神经网络的研究进入了反思期。很多献身于神经网络研究的科学家的研究结果很难得到发表，而且是散布在各种杂志之中，使得不少有意义的研究成果即使发表了，也很难被同行看到，著名的BP算法的研究就是一个典型的例子。
- 反思期的到来揭示了人类的认识规律：认识->实践->再认识。

二、神经网络的发展历史

第二次高潮期（1983~1990）：

- 1982年，物理学家Hopfield提出了一种具有联想记忆、优化计算能力的递归网络模型，即Hopfield 网络。
- 1984年，Hopfield设计并实现了后来被人们称为Hopfield网络的电路。较好地解决了著名的TSP组合优化问题，找到了最佳解的近似解，引起了较大的轰动。
- Hopfield这两项成果分别于1982和1984年发表在美国科学院院刊上，标志着神经网络研究第二高潮期的到来。
- 1986年，Rumelhart 等编辑的著作《Parallel Distributed Proceesing:Explorations in the Microstructures of Cognition》报告了反向传播算法

二、神经网络的发展历史

第二次高潮期（1983~1990）：

- 1985年，Hinton、Sejnowsky、Rumelhart等人所在的并行分布处理(PDP)小组在Hopfield网络中引入了随机机制，提出Boltzmann机。
- 1986年，PDP小组的Rumelhart等研究者重新独立地提出了多层神经网络的学习算法—BP算法，较好地解决了多层神经网络的学习问题。
- 1987年，IEEE 在美国加州圣地亚哥召开第一届神经网络国际会议(ICNN)，宣告了国际神经网络协会正式成立，掀起了人类向生物学习、研究及应用神经网络的新热潮。
- 1990年，国内首届神经网络大会在北京举行。

二、神经网络的发展历史

再认识与应用期（1991~2012）：

- 90年代初，伴随**统计学习理论**和**SVM**的兴起，神经网络由于理论不够清楚，试错性强，难以训练，再次进入低谷。
- 2006年，Hinton提出了**深度信念网络(DBN)**，通过“预训练+微调”使得深度模型的最优化变得相对容易。
- **2012年**，Hinton组参加**ImageNet**竞赛，使用**CNN**模型以**超过第二名10个百分点**的成绩夺得当年竞赛的冠军。

二、神经网络的发展历史

第三次浪潮（2012~）：

- 2016年3月谷歌人工智能AlphaGo在围棋比赛中4：1战胜世界围棋冠军李世石；2017年5月，又在浙江乌镇的围棋峰会上，以3：0完胜我国世界排名第一棋手柯洁。
- 2017年1月6日，最强大脑第四季引入“人机大战”模式，百度研发的人工智能机器人“小度”作为特别选手参赛，并在比赛中战胜了最强大脑的队长——王峰。
- 伴随云计算、大数据时代的到来，计算能力的大幅提升，使得深度学习模型在计算机视觉、自然语言处理、语音识别等众多领域都取得了较大的成功。

二、神经网络的发展历史

- ALVINN (Pomerleau 1993)
 - 自动驾驶
 - 输入：30x32 图片
 - ✓ 从下面进入节点的行是它的输入
 - 隐藏单元
 - ✓ 它们的输出仅在网络内可用，不能作为网络输出的一部分
 - 输出单元
 - ✓ 每个输出单元对应一个特定的转向方向，输出值决定驱动动作

二、神经网络的发展历史

- ALVINN (Pomerleau 1993)

相机照片

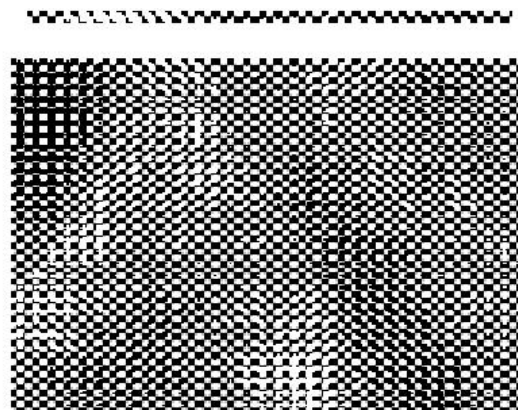
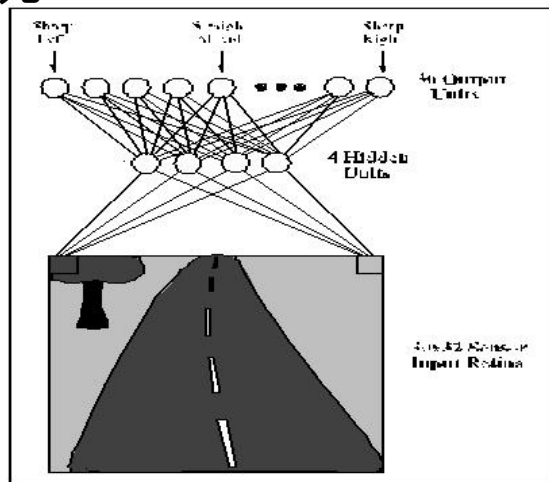


进入4个隐藏层单元的
30x32个权值

30个输出单元

4 个隐藏层
单元

30x32 像素
作为输入

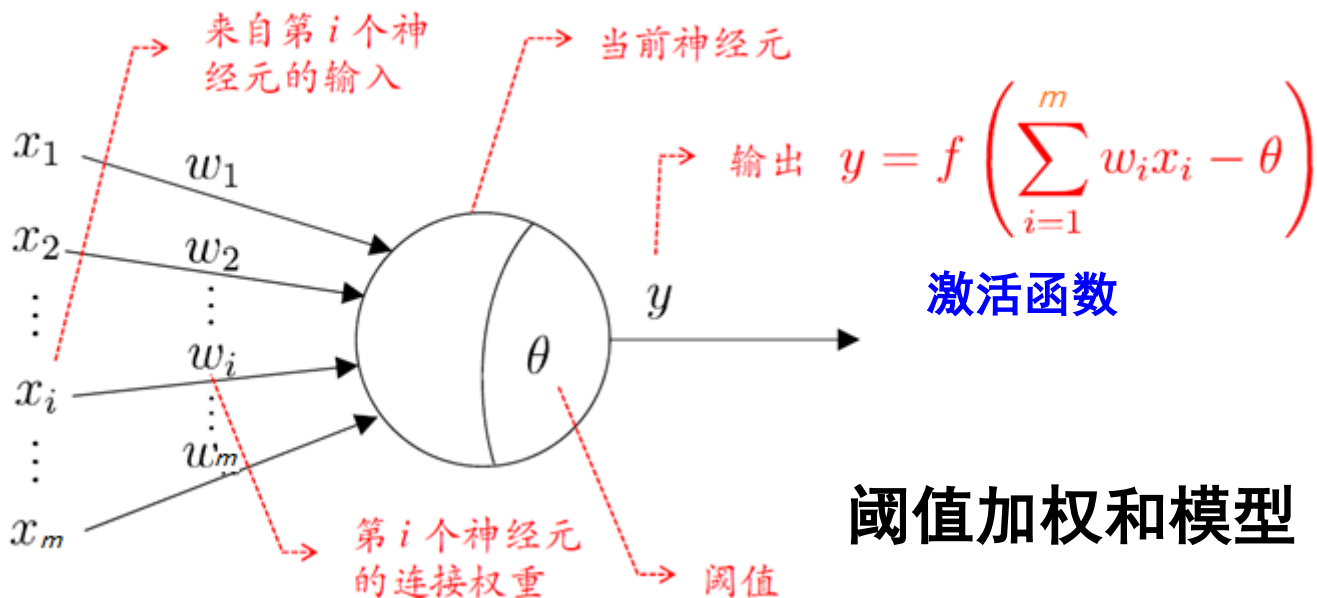


在公路上时速70英里

三、M-P神经元模型

- 最基本的信息处理单元：M-P神经元模型

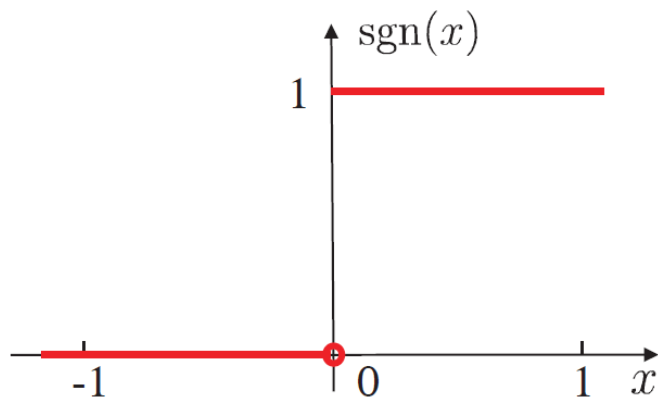
- **输入**：来自其他 m 个神经元传递过来的输入信号
- **处理**：输入信号通过带权重的连接进行传递，神经元接收到**总输入值**将与神经元的**阈值**进行比较
- **输出**：通过**激活函数**的处理以得到输出



三、M-P神经元模型

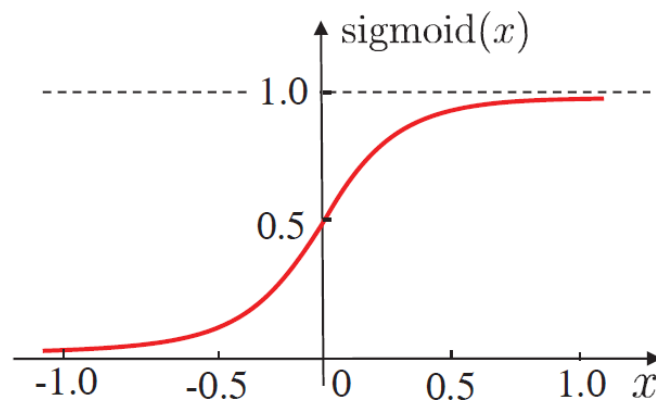
激活函数

- 理想激活函数是阶跃函数，0表示抑制神经元而1表示激活神经元
- 阶跃函数具有不连续、不光滑等不好的性质，常用的是 Sigmoid 函数



$$\text{sgn}(x) = \begin{cases} 1, & \text{if } x \geq 0; \\ 0, & \text{if } x < 0. \end{cases}$$

(a) 阶跃函数



$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

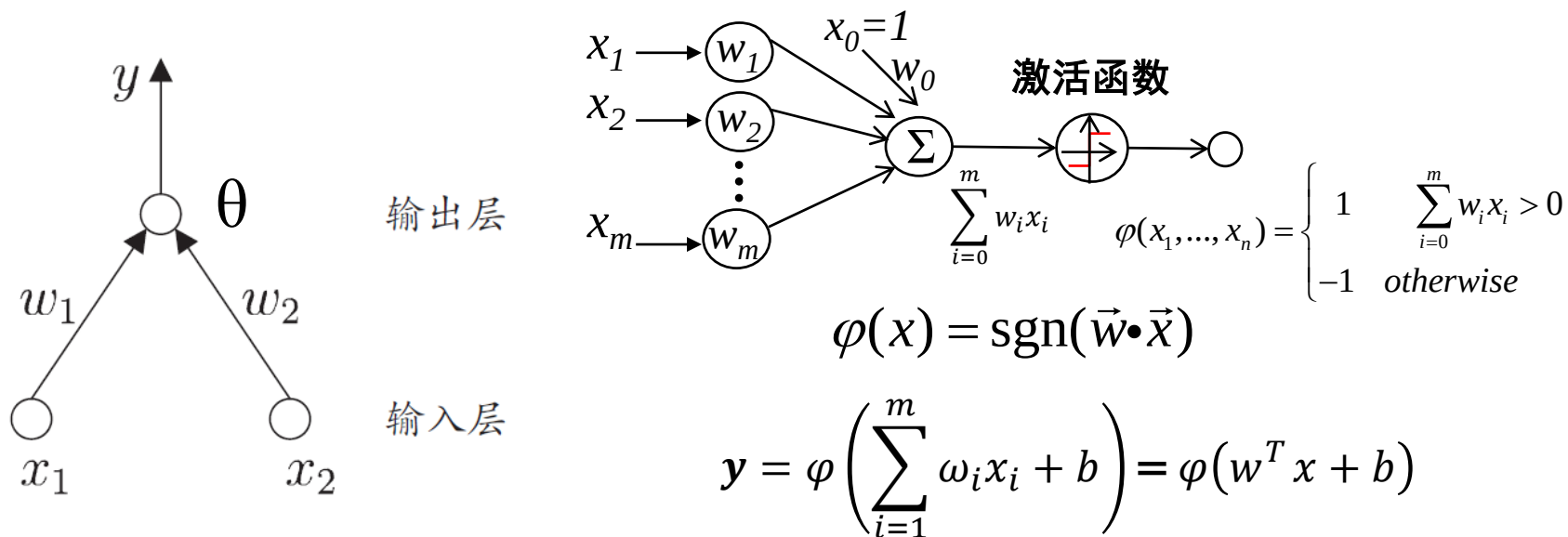
(b) Sigmoid 函数

典型的神经元激活函数

三、感知器模型

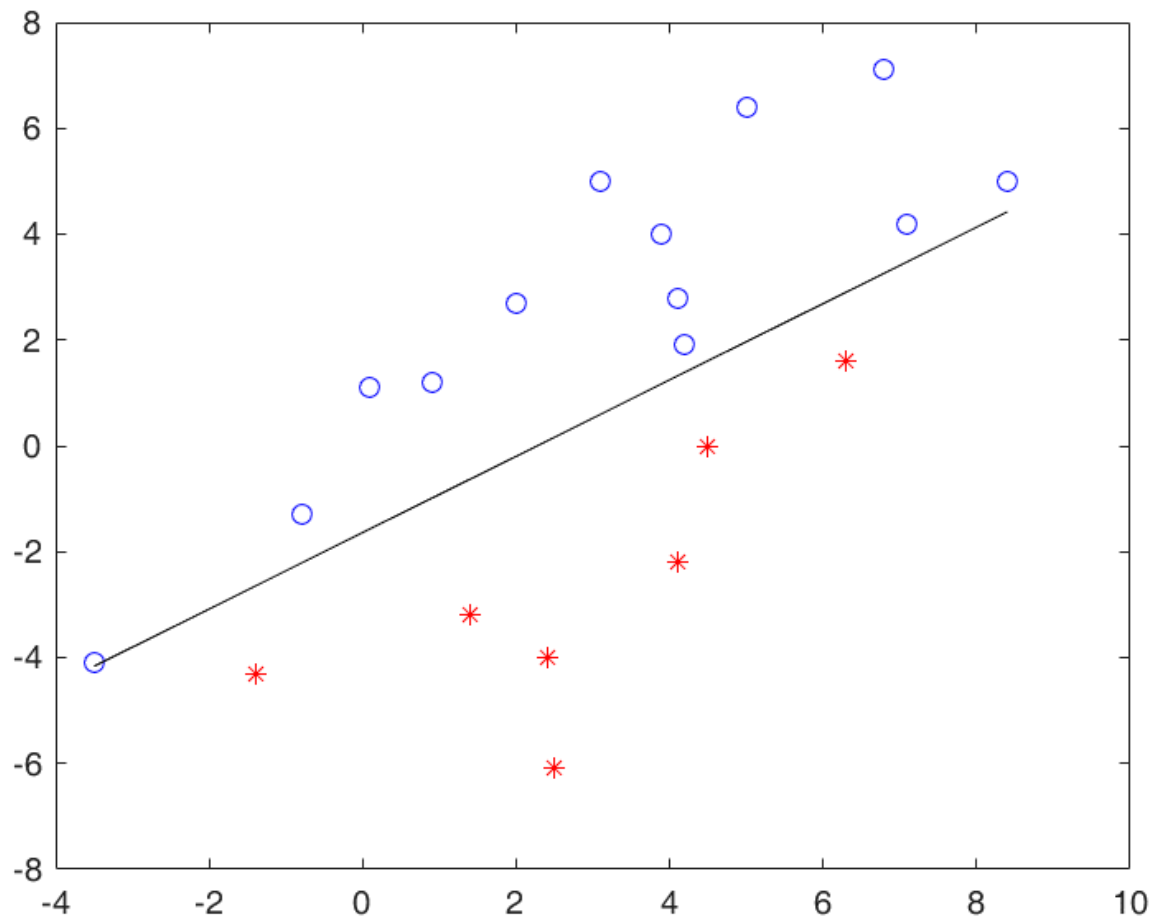
单层感知器的定义

- 单层感知器只拥有一层M-P神经元，即只包含输入层和输出层，输入层接受外界输入信号后传递给输出层，输出层是M-P神经元，进行激活处理。
- 比如，下图所示的M-P神经元模型，就是一个最简单的单层感知器，输出层只有一个神经元。



三、感知器模型

感知器算法演示：



三、感知器模型

- 感知器能够容易地实现逻辑与、或、非运算(布尔)

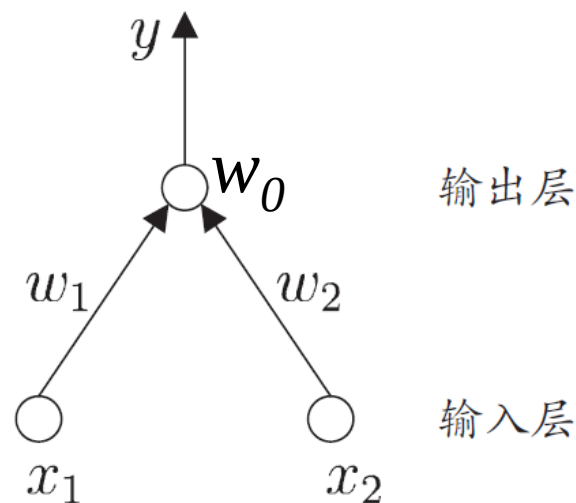
- **与** $x_1 \wedge x_2$: 令 $w_1=w_2=1$, $w_0=2$,
则 $y = \varphi(1*x_1+1*x_2-1.5)$, 仅当
 $x_1=x_2=1$ 时, $y=1$ 。

- **或** $x_1 \vee x_2$: 令 $w_1=w_2=1$, $w_0=0.5$,
则 $y = \varphi(1*x_1+1*x_2-0.5)$, 仅当
 $x_1=1$ 或者 $x_2=1$ 时, $y=1$ 。

- **非** $\neg x_1$: 令 $w_1=-0.6$, $w_2=0$, $w_0=-0.5$,
 $y = \varphi(-0.6*x_1+0*x_2+0.5)$, 此时:

$x_1=1$ 时, $y=0$;

$x_1=0$ 时, $y=1$ 。



三、感知器模型

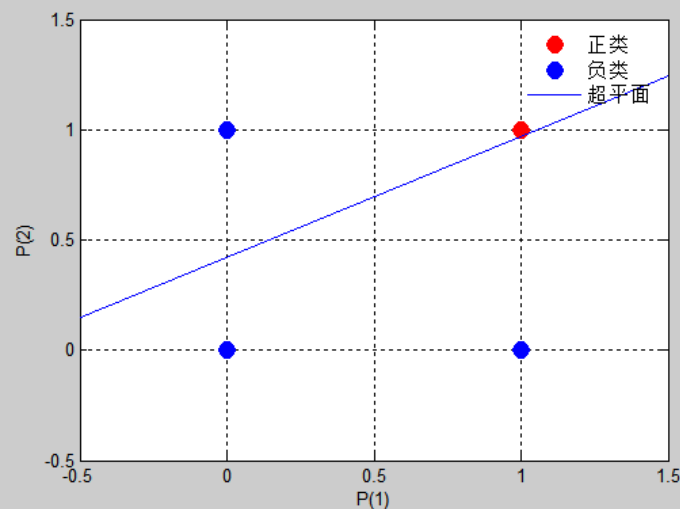
感知器的表征能力

- 感知器可以看作是m维实例空间(点空间)中的一个超平面决策面
 - 超平面一侧的正实例输出+1, 反例在另一边输出-1
- 应用的前提基础: 线性可分的样例集合
- 决策超平面的方程: $\vec{w} \cdot \vec{x} = 0$
- 单层感知器可以表示所有的原子布尔函数
 - 布尔函数:m-of-n函数(和或)
 - AND OR NAND NOR
- 一些布尔函数无法用单层的感知器表示
 - 线性不可分的布尔函数(例如异或)是不可表示的

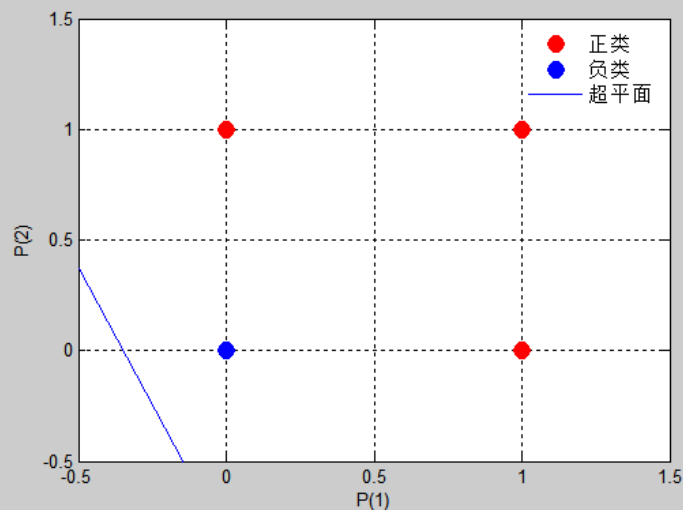
三、感知器模型

单层感知器求解与、或、非问题

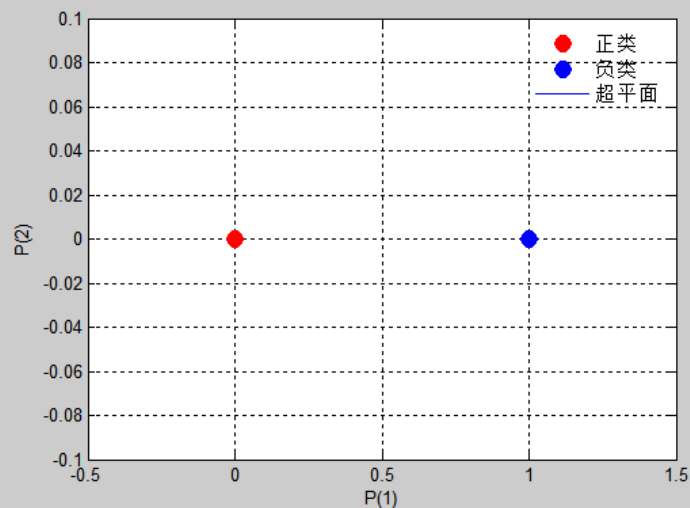
与问题-感知机学习 $-w_{int}=[0.34 \ 0.62 \ -0.26]$



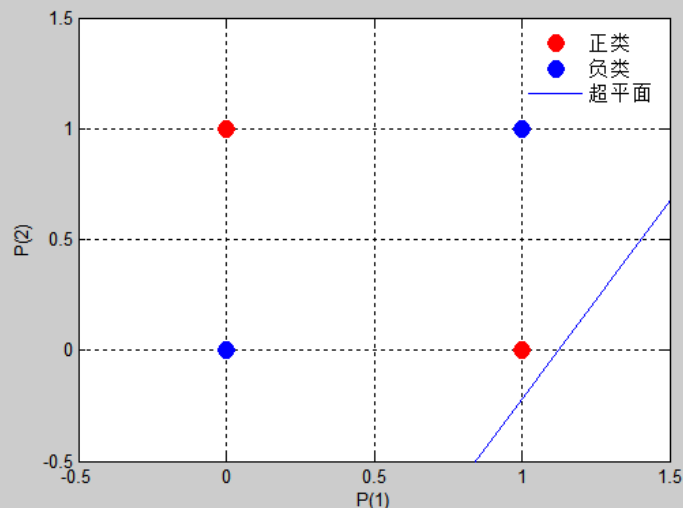
或问题-感知机学习 $-w_{int}=[0.71 \ 0.29 \ -0.25]$



非问题-感知机学习 $-w_{int}=[-0.23 \ 0.17 \ -0.50]$

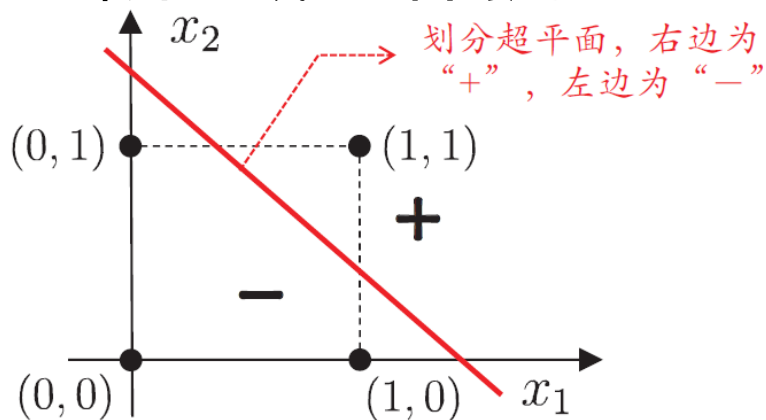


异或问题-感知机学习 $-w_{int}=[-0.42 \ 0.23 \ -0.47]$

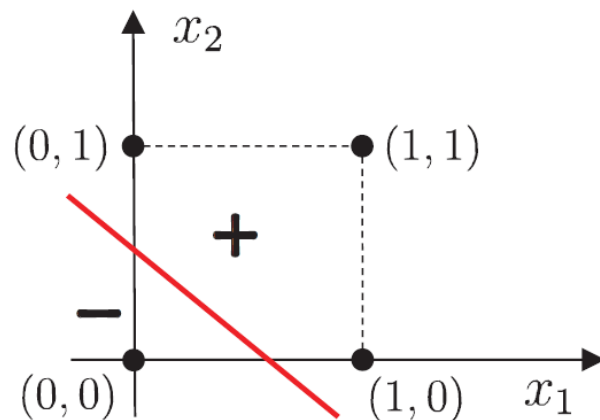


三、感知器模型

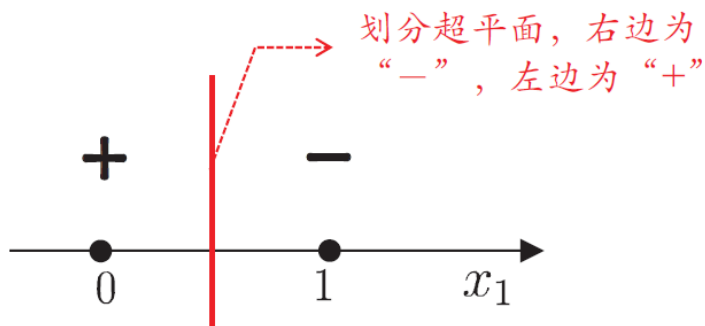
感知器求解异、或、非问题



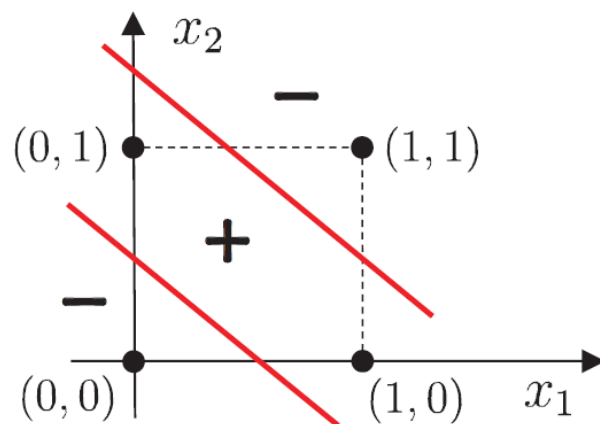
(a) “与”问题 ($x_1 \wedge x_2$)



(b) “或”问题 ($x_1 \vee x_2$)



(c) “非”问题 ($\neg x_1$)



(d) “异或”问题 ($x_1 \oplus x_2$)

线性可分的“与”“或”“非”问题与非线性可分的“异或”问题

三、感知器模型

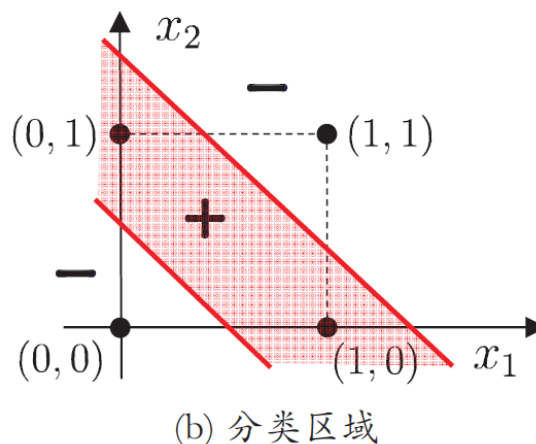
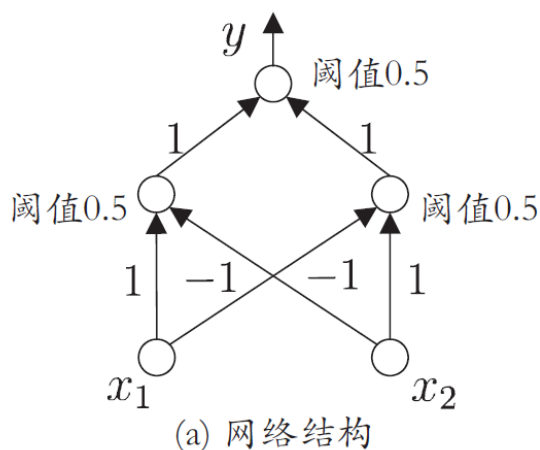
单层感知器的学习能力：

- 尽管单层感知机能够非常容易地实现逻辑与、或、非运算，但其学习能力还是非常有限。
- 事实上，逻辑与、或、非运算问题都是线性可分的问题。可以证明[Minsky and Papert, 1969]，若二分类问题是线性可分的，即存在一个线性超平面能将它们分开，则感知机的学习过程一定会收敛，否则感知机的学习过程将会发生振荡，甚至不能解决异或这样简单的非线性可分问题。
- 要解决非线性可分问题，就需要使用多层功能神经元。比如两层感知机，输入层与输出层之间的一层神经元，被称为隐含层，隐含层和输出层神经元都是拥有激活函数的功能神经元。

三、感知器模型

多层感知机

- 解决异或问题的两层感知机



能解决异或问题的两层感知机

- 输出层与输入层之间的一层神经元，被称之为**隐层或隐含层**，隐含层和输出层神经元都是具有激活函数的功能神经元
- 每个逻辑布尔函数都可以用只有两层深度的感知器网络来表示

三、感知器模型

感知器训练法则

- 目的是学习由多个单元互连的网络，但如何学习和理解单个感知器的权
- 学习任务
 - 确定一个权向量，使感知器为给定的训练样例输出正确的+1或-1
- 学习算法
 - 感知器训练法则
 - Delta 法则
 - 多层网络学习优化算法

两者都保证收敛到一个可接受的权向量

三、感知器模型

单层感知器的训练规则（感知器学习法则）：

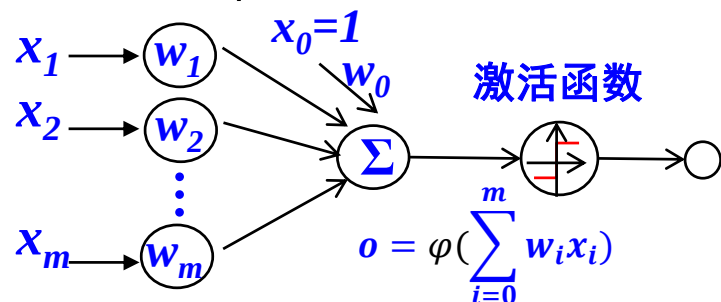
- 给定训练数据集，单层感知器的权重 $w_i (i=1,2,\dots, m)$ 与阈值 θ 可以很容易通过学习得到。
- 在单层感知器中，神经元的阈值 θ 可看作一个固定输入为 -1 的“哑结点”所对应的连接权重 w_0 ，这样，权重和阈值的学习就统一为权值的学习了。
- 对训练样例 (x, t) ，若当前感知器的输出为 o ，则感知器权重调整规则为：

$$w_i \leftarrow w_i + \Delta w_i$$

$$\Delta w_i = \eta (t - o) x_i$$

其中 $\eta \in (0,1)$ 为学习率。

- 若感知器对训练样例 (x,y) 预测正确，则感知器不发生变化；否则，根据错误程度进行权重的调整。



三、感知器模型

感知器学习法则

- 感知器权重调整规则

$$w_i \leftarrow w_i + \Delta w_i$$

$$\Delta w_i = \eta (t - o) x_i$$

➤ $t = c(x)$: 目标值

➤ o : 感知器输出

➤ η : 很小的常数 (e.g. 0.1), 调整权值更新的幅度, 叫做**学习率**

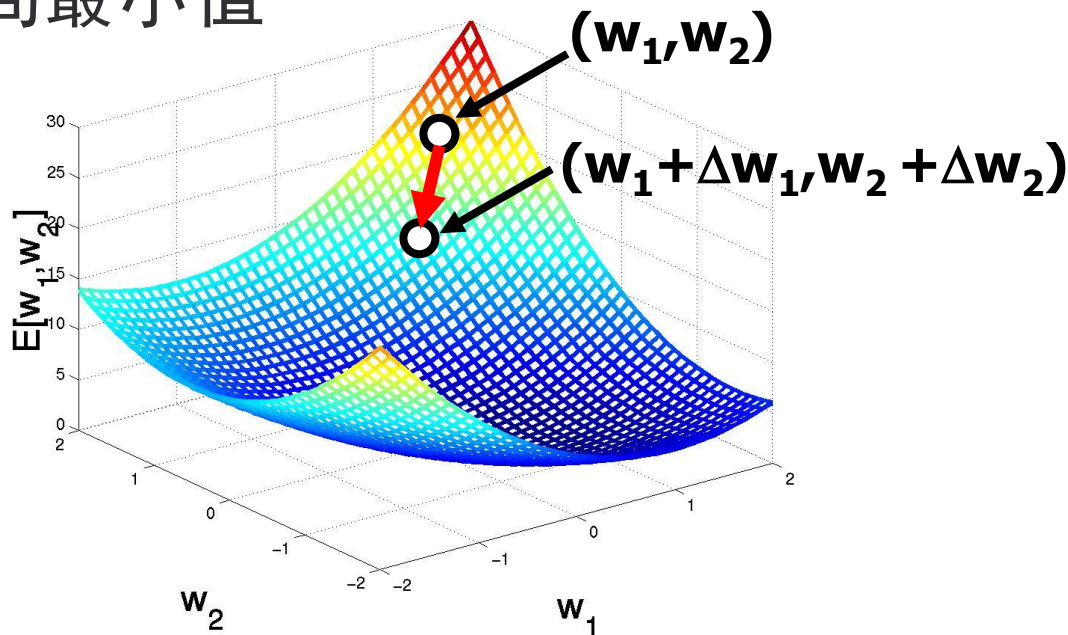
- 如果输出是**正确**的($t = o$), w_i **不会改变**
- 如果输出是**错误**的($t \neq o$), w_i 会改变
 - 使感知器的输出为新的权值更加接近 t
- 学习法则的**收敛条件** (Minsky&Papert 1969)
 - 给定的训练数据是线性可分的
 - η 是足够小的
 - 如果数据不是线性可分的, 就不能保证收敛性

四、梯度下降法则和Delta法则

- Delta 法则克服了感知器规则的困难
 - 如果训练的例子不是线性可分的，Delta规则会向目标概念的最佳拟合近似值收敛
- Delta法则隐含的**关键思想**
 - 使用**梯度下降法**搜索可能的权向量的假设空间，以找到**最佳拟合**训练实例的权值向量
- 理解 Delta 法则
 - Delta规则的任务：训练一个**无阈值感知器**（线性单元）
$$o(\vec{x}) = \vec{w} \cdot \vec{x}$$
 - 为衡量假设(权向量)对训练样例拟合的训练误差指定一个**度量**
$$E(\vec{w}) = \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2$$
 - 为什么要找到使得E最小的假设呢？
 - 贝叶斯学习为选择如何选择E给出了**贝叶斯论证**。在一定的条件下，对于给定训练数据使得E最小化的假设也是H中最可能的假设

四、梯度下降法则（可视化假设空间）

- 假设空间 H 是 $[w_1, w_2]$ 的曲面。对于线性单元，这个误差曲面有一个全局最小值



- 梯度下降搜索

➤ 确定一个使 E 最小化的权向量

- 从任意的初始权向量 w 开始，以很小步伐反复修改这个向量
- 每一步沿误差曲面产生最陡峭下降的方向修改权向量 w
- 迭代直到得到全局最小误差点

四、梯度下降法则（梯度下降规则的推导）

- 我们怎样才能找到最陡峭下降的方向？

- 计算E相对向量 $\vec{w}$ 的各个分量的导数来得到 $\nabla E(\vec{w})$

$$\nabla E(\vec{w}) = \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \dots, \frac{\partial E}{\partial w_m} \right]$$

- 当**梯度**被解释为**权空间的一个向量**时，它确定了使E最陡峭上升的方向

- 函数某点梯度的方向是方向导数中取到最大值的方向，梯度的模是方向导数的最大值
- 这个向量的反方向就是下降最快的方向

- **梯度下降训练规则**

$$\vec{w} \leftarrow \vec{w} + \Delta \vec{w}$$

$$\Delta \vec{w} = -\eta \nabla E(\vec{w})$$

四、梯度下降法则

梯度下降法则的例子：

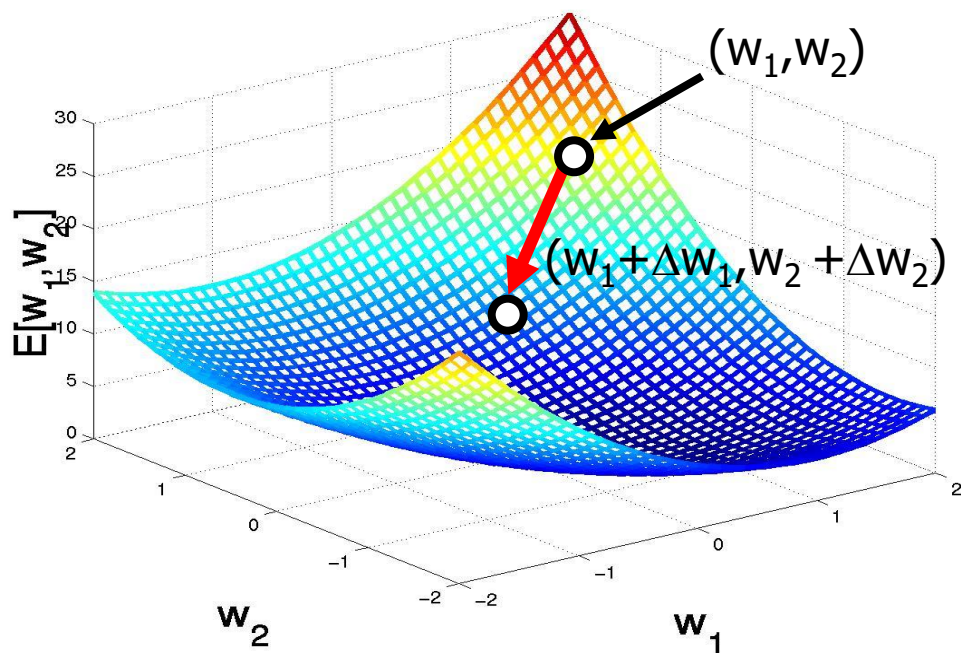
$$D=\{<(1,1),1>,<(-1,-1),1>,<(1,-1),-1>,<(-1,1),-1>\} \quad E(\vec{w}) = \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2$$

Gradient:

$$\nabla E(\vec{w}) = \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \dots, \frac{\partial E}{\partial w_m} \right]$$

$$\Delta w = -\eta \nabla E(\vec{w})$$

$$\begin{aligned} \frac{\partial E}{\partial w_i} &= \frac{\partial}{\partial w_i} \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2 \\ &= \frac{1}{2} \sum_{d \in D} \frac{\partial}{\partial w_i} (t_d - o_d)^2 \\ &= \frac{1}{2} \sum_{d \in D} 2(t_d - o_d) \frac{\partial}{\partial w_i} (t_d - o_d) \\ &= \sum_{d \in D} (t_d - o_d) \frac{\partial}{\partial w_i} (t_d - \vec{w} \cdot \vec{x}_d) \end{aligned}$$



$$\nabla E(\vec{w}_i) = \sum_{d \in D} (t_d - o_d) (-x_{id})$$

$$\Delta w_i = -\eta \nabla E(\vec{w}_i) = \eta \sum_{d \in D} (t_d - o_d) x_{id}$$

四、梯度下降法则

- 收敛是有保证的
 - 由于误差曲面只包含一个全局最小值，无论训练实例是否线性可分，只要给定一个足够小的学习率，该算法将收敛到一个具有最小误差的权向量 $\vec{w}$
- 对学习率一个常见改进策略
 - 随着梯度下降步数的增加，学习率的值逐渐降低
- 梯度下降法则是一种重要的通用学习范型
 - 可以用于搜索一个庞大的或无限的假设空间
 - 假设空间应满足以下条件（任何情况）
 - ✓ 假设空间包含连续参数化假设（如线性单元权值）
 - ✓ 误差对于这些假设参数可微分

四、梯度下降法则

Gradient-Descent(*training_examples*, η)

- 训练样本中每个训练样例形式为序偶 $\langle \vec{x} = (x_1, \dots, x_m), t \rangle$, 其中 $\vec{x}$ 是输入向量, t 是目标输出值, η 是学习率(如0.1)
- 初始化每个 w_i 为某个小的随机值
- 在满足终止条件之前, 执行
 - 初始化每个 Δw_i 为 0
 - 对每一个在训练样例中的序偶 $\langle \vec{x}, t \rangle$, 执行
 - ✓ 将实例 $(x_1, \dots, x_m)$ 输入到线性单元并计算输出 o
 - ✓ 对线性单元的每一个权 w_i , 执行

$$\Delta w_i \leftarrow \Delta w_i + \eta (t - o) x_i$$

- 对线性单元的每一个权 w_i , 执行

$$w_i = w_i + \Delta w_i$$

四、梯度下降法则（梯度下降的随机近似）

- 应用梯度下降法的**关键实践难点**
 - 收敛速度可能很慢（如几千步梯度下降）
 - 如果误差曲面中存在多个局部极小值，则不能保证该过程能够找到全局最小值
- **随机近似**（增量/随机梯度下降, **Stochastic Gradient Decent (SGD)**)
 - GD训练法则对D中**所有训练样例**求和后计算权值更新，SGD**根据每一个单独样例的误差增量**计算权值更新，得到近似的梯度下降搜索。
 - 修改的训练规则：只在迭代更新每一个样例时更新权值

$$\Delta w_i = \eta \sum_{d \in D} (t_d - o_d) x_{id} \quad \textbf{(GD)}$$

$$\Delta w_i = \eta (t_d - o_d) x_i \quad \textbf{(SGD)}$$

四、梯度下降法则（梯度下降的随机近似）

Stochastic gradient(*training_examples*, η)

- 训练样本中每个训练样例形式为序偶 $\langle \vec{x} = (x_1, \dots, x_m), t \rangle$, 其中 $\vec{x}$ 是输入向量, t 是目标输出值, η 是学习率(如0.1)
- 初始化每个 w_i 为某个小的随机值
- 在满足终止条件之前, 我们执行
 - 初始化每个 Δw_i 为 0
 - 对每一个在训练样例中的序偶 $\langle \vec{x}, t \rangle$, 执行
 - 将训练实例 $(x_1, \dots, x_m)$ 输入到线性单元, 并计算输出 o
 - 对线性单元的每一个权值 w_i , 计算

$$w_i \leftarrow w_i + \eta (t - o) x_i$$

四、梯度下降法则（梯度下降的随机近似）

- 理解随机梯度下降

- 为每个训练样例 d 创建一个不同的误差函数

$$\text{GD: } E(\vec{w}) = \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2 \quad \text{SGD: } E_d(\vec{w}) = \frac{1}{2} (t_d - o_d)^2$$

- 当在迭代所有训练样例时，这些权重更新的序列给出了对原始误差函数 $E(\vec{w})$ 的梯度下降的一个合理近似

$$\text{GD: } \Delta w_i = \Delta w_i + \eta (t - o) x_i \quad \text{SGD: } \Delta w_i = \eta (t - o) x_i$$

- SGD以任意程度接近于真实的GD

- 前提：当学习速率足够小

- 标准GD和SGD的关键区别（书上讲3点）

- 权值更新的训练样例数量：GD（汇总所有数据误差）SGD（每个实例）

- 每个权值更新的步长：GD更大

- SGD逃离局部极小值的机会更大

- 随机梯度下降法和标准梯度下降法在实践中都是常用方法

四、梯度下降法则（梯度下降的随机近似）

- Delta法则也称LMS法则、Adaline法则、Windrow-Hoff法则

Delta法则和感知器训练法则的差值(相同?)

Delta: $\Delta w_i = \eta (t - o) x_i$ $o(\vec{x}) = \vec{w} \cdot \vec{x}$ (线性单元)

感知器: $\Delta w_i = \eta (t - o) x_i$ $o'(\vec{x}) = \varphi(\vec{w} \cdot \vec{x})$ (阈值输出)

- Delta也可以用来训练带阈值感知器单元
 - 如果训练非阈值输出 o 能完美地拟合这些值，则阈值输出 o' 也会拟合它们
 - 即使目标值不能被完美拟合，只要线性单位的输出 o 有正确的符号，有阈值的 o' 值会正确地拟合目标值 ± 1
- Delta法则将学习使线性单元输出 o 的误差最小化的权重，这些权重不能保证也使有阈值输出 o' 的误分类样本数最小化。

感知器法则 v. s. Delta法则

- 感知器法则和Delta法则的**关键区别**

- 感知器法则根据**阈值化的感知器**输出中的误差更新权重
- Delta法则根据**非阈值化线性组合**的误差来更新权重

- **收敛属性的差异**

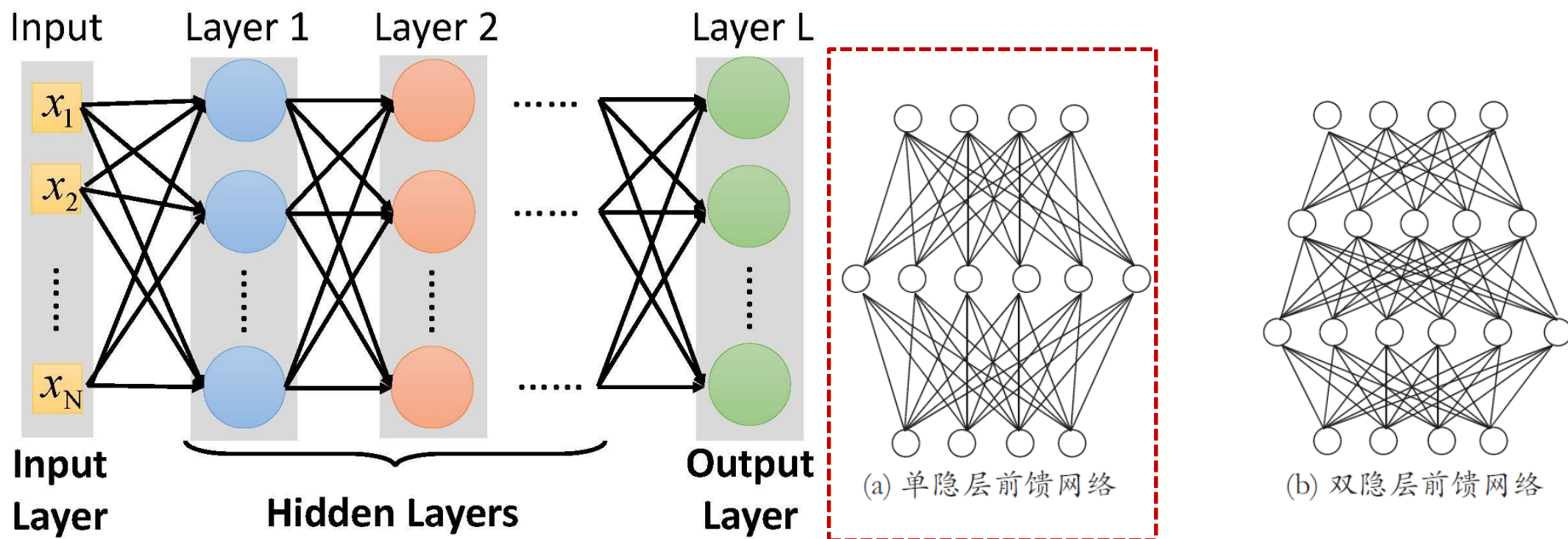
- 假设**训练样例线性可分**，感知器训练法则经过**有限次迭代**后收敛到一个能够**理想分类**训练数据的假设
- Delta法则**渐进收敛**到最小误差假设，可能需要**极长时间**，而**无论训练数据是否线性可分**均可以收敛

- 学习权向量的第三种算法是**线性规划**

- 线性规划是求解线性不等式方程组的一种通用方法
- 这种方法只有在训练实例线性可分的情况下才有解
- 不能扩展到训练多层网络，但基于Delta法则的梯度下降法可以很容易地推广到多层网络

五、多层前馈神经网络

多层前馈神经网络能够表示种类繁多的非线性曲面：



多层前馈网的结构图

五、多层前馈神经网络

多层前馈神经网络的定义：

- 多层前馈神经网络又名多层前馈全互连接网

- **多层**是指：除了**输入层**和**输出层**以外，还存在一个或者多个**隐含层**。每层神经元与下一层神经元全互联，神经元之间**不存在同层连接**也不存在跨层连接。
- **前馈**是指：外界信号从输入层，隐含层与输出层神经元对信号进行加工，最终结果由输出层神经元输出，**不存在信号的逆向传播**。
- **全连接神经网络**是指：任意两个相邻层之间的神经元全部互相连接，每层神经元与下层神经元**全互连接**。
- **学习**：根据训练数据来调整神经元之间的“**连接权**”以及每个功能神经元的“**阈值**”
- **多层网络**：包含隐层的网络

五、多层前馈神经网络

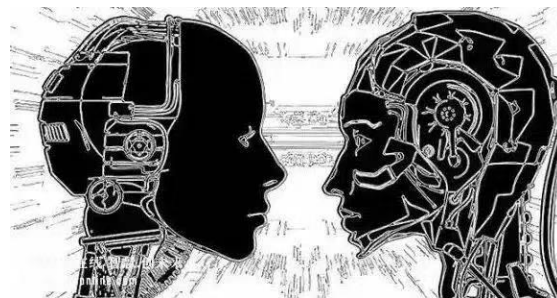
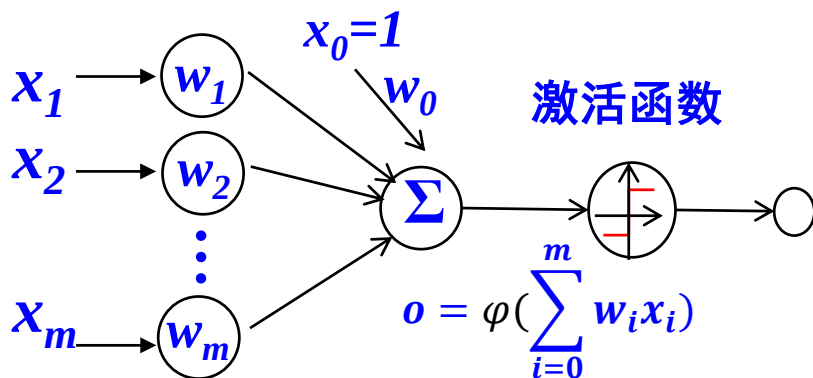
多层前馈神经网络的定义：

- 输入层神经元接收外界输入，隐含层与输出层神经元对信号进行加工，最终结果由输出层神经元输出。也就是说，输入层神经元只是接受输入，不进行函数处理，隐含层与输出层才进行函数处理。
- 神经网络参数学习就是根据训练数据来调整神经元之间的连接权以及每个功能神经元的阈值。换言之，神经网络学到的东西都存放在连接权与阈值中。

五、多层网络(可微阈值单元)

可微阈值单元

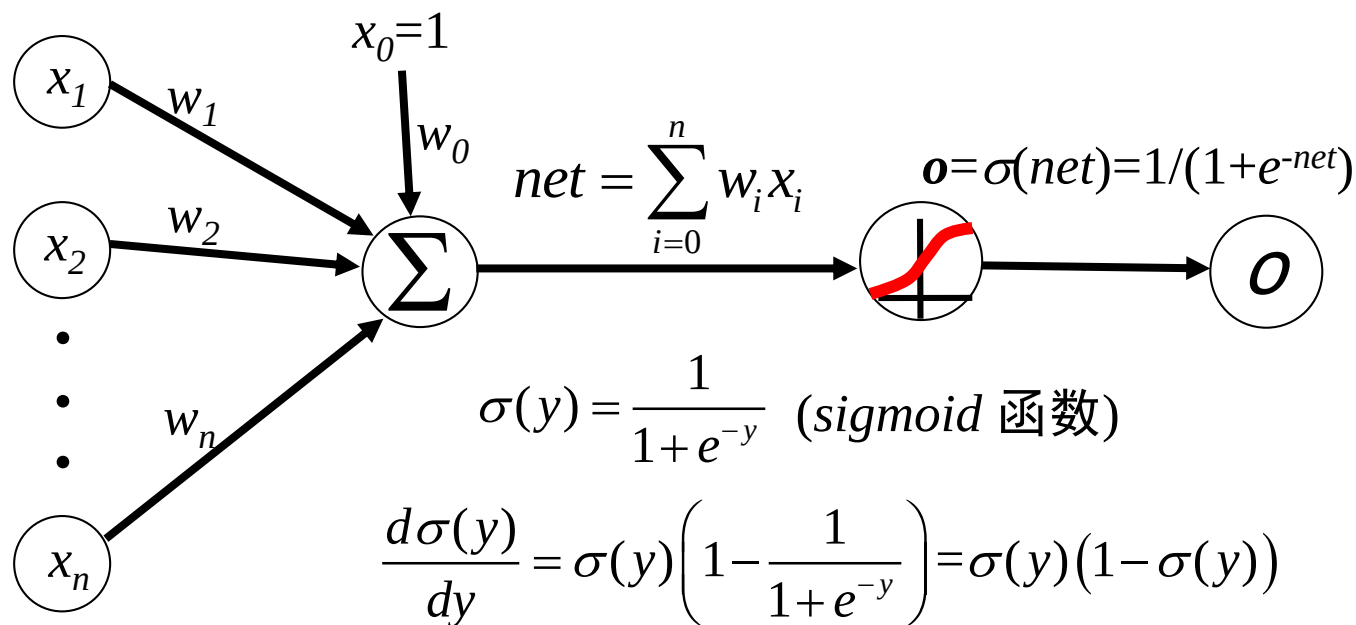
- 什么类型的单元作为构建多层网络的基础？
 - 能够表征高度非线性的函数
 - 适用于梯度下降（可微分）
- 所有单元（神经元）应该是
 - 输出是输入的非线性函数
 - 输出是输入的可微函数



五、多层网络(可微阈值单元)

可微阈值单元

Sigmoid 单元



梯度下降法则推导 sigmoid 函数

$$E(\vec{w}) = \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2$$
$$\frac{\partial E}{\partial w_i} = \sum_{d \in D} (t_d - o_d) \frac{\partial}{\partial w_i} (t_d - o_d)$$
$$= - \sum_{d \in D} (t_d - o_d) o_d (1 - o_d) x_i$$

五、多层网络(可微阈值单元)

可微阈值单元 --Sigmoid 单元

- Sigmoid 单元的特点
 - 平滑的微分阈值函数
 - Sigmoid 函数 (Logistic函数, Squashing函数)
 - 输出范围为0到1
 - 单调增加
 - 它的导数很容易用输出来表示
- 其他易计算导数的可微函数
 - sigmoid 的陡峭性(e^{-ky}), k 是正的常数
 - 双曲正切 $\tanh$ $\frac{e^x - e^{-x}}{e^x + e^{-x}}$

五、反向传播算法

多层前馈神经网络的学习

- 多层前馈神经网络的表达能力比单层感知机要强得多。要学习多层前馈神经网络，单层感知机的学习规则是远远不够的，需要更强大的学习算法。
- 误差逆传播（Error Back Propagation，简称BP），也称作反向传播算法，该算法就是其中最杰出的代表，它是迄今最成功的神经网络学习算法。
- 现实任务中使用神经网络时，大多是在使用BP算法进行训练。需要指出的是，BP算法不仅可用于多层前馈神经网络，还可用于其他类型的神经网络，比如递归神经网络。但通常说BP网络的时候一般是指用BP算法训练的多层前馈神经网络。

五、反向传播算法

- 学习多层网络的权重

- 采用梯度下降方法试图最小化网络输出值和目标值之间的误差平方
- 重新定义E以对所有网络输出单元上的误差求和

$$E(\vec{w}) = \frac{1}{2} \sum_{d \in D} \sum_{k \in \text{outputs}} (t_{kd} - o_{kd})^2$$

- 反向传播算法的难点

- 搜索巨大的假设空间
- 误差曲面可以有多个局部极小值

- 实践中发现反向传播算法总可以产生出色的结果

五、反向传播算法

误差逆传播算法是最成功的训练多层神经网络的学习算法。

- 给定训练集给 $D=\{\{x_i, y_i\}, x_i \in R^d, y_i \in R^l, (i = 1, 2, \dots, m)\}$, 即输入示例由 d 个属性描述, 输出 l 维实值向量。
- 为方便讨论, 给定一个拥有 d 个输入神经元, l 个输出神经元, q 个隐层神经元的多层前向前馈网络结构。

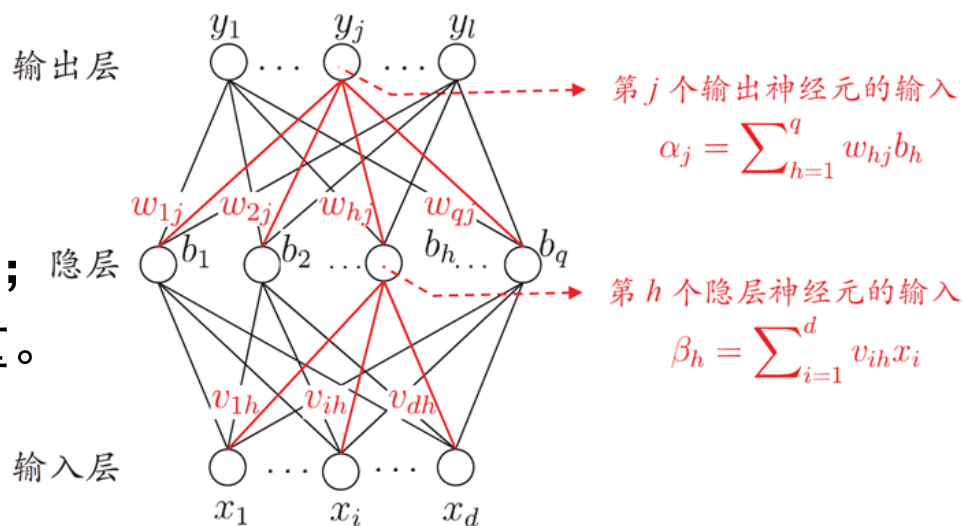
● 记号:

θ_j : 输出层第 j 个神经元阈值;

γ_h : 隐含层第 h 个神经元阈值;

v_{ih} : 输入层与隐层之间的连接权重;

w_{hj} : 隐层与输出层之间的连接权重。



五、反向传播算法

对于样例 (x_k, y_k) , 假设网络的实际输出为 $\hat{y}$

● 前向计算

step1: $b_h = \sigma(\beta_h - \gamma_h)$, $\beta_h = \sum_{i=1}^d v_{ih} x_i$

step2: $\hat{y}_j^k = \sigma(\alpha_j - \theta_j)$, $\alpha_h = \sum_{i=1}^q w_{hj} b_h$

step3: $E_k = \frac{1}{2} \sum_{j=1}^l (\hat{y}_j^k - y_j^k)^2$

● 参数数目

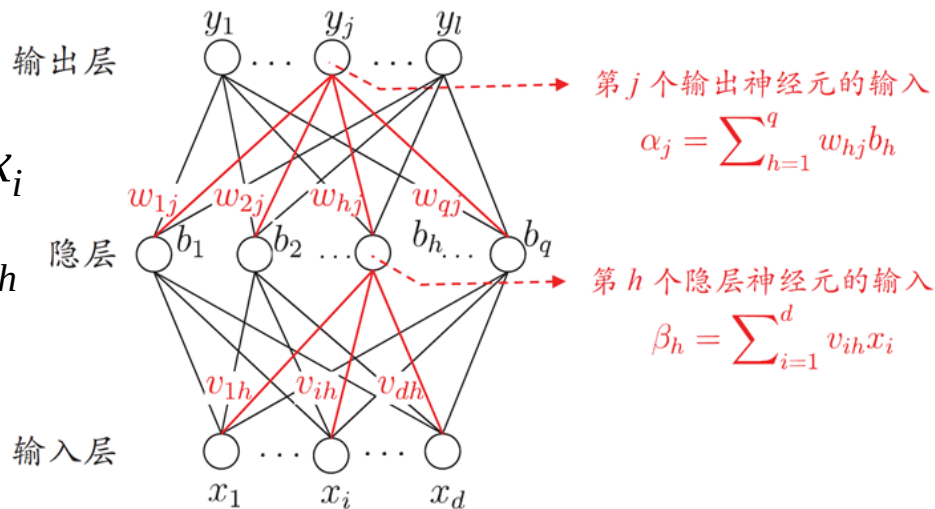
权重: v_{ih}, w_{hj} 阈值: θ_j, γ_h ($i=1, \dots, d; h=1, \dots, q; j=1, \dots, l$)

因此, 网络中需要 $(d+l+1)q+l$ 个参数需要优化

● 参数优化

- BP是一个迭代学习算法, 在迭代的每一轮中采用广义的感知机学习规则对参数进行更新估计, 任意的参数 v 的更新估计式为

$$v \leftarrow v + \Delta v$$



五、反向传播算法

- BP算法基于梯度下降策略, 以目标的负梯度方向对参数进行调整, 对误差 E_k , 给定学习率 η

$$E_k = \frac{1}{2} \sum_{j=1}^l (\hat{y}_j^k - y_j^k)^2$$

$$\Delta w_{hj} = -\eta \frac{\partial E_k}{\partial w_{hj}}$$

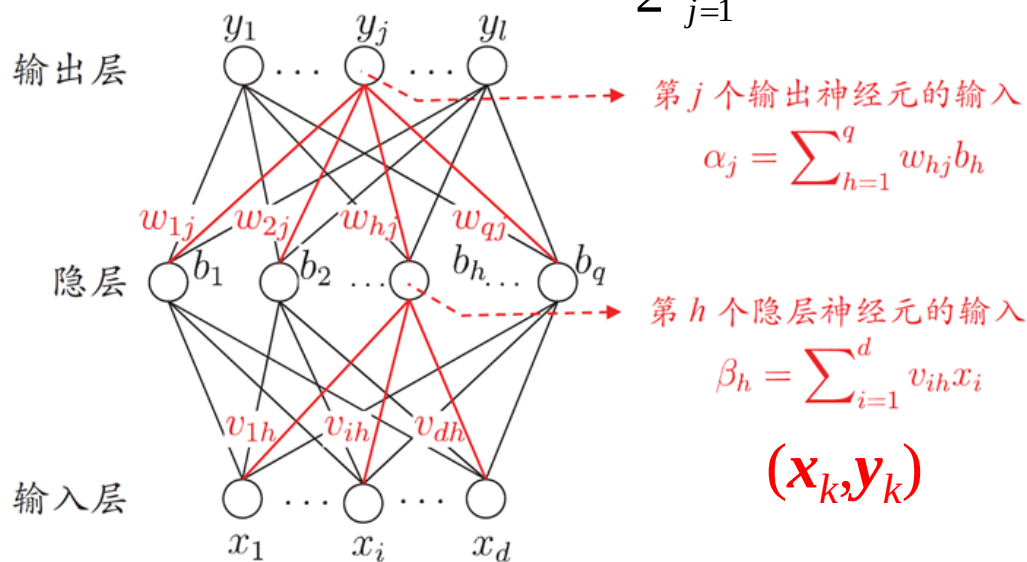
$$\frac{\partial E_k}{\partial w_{hj}} = \frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \alpha_j} \cdot \frac{\partial \alpha_j}{\partial w_{hj}}$$

$$g_j = \frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \alpha_j}$$

$$= -(y_j^k - \hat{y}_j^k) \sigma'(\alpha_j - \theta_j)$$

$$= -\hat{y}_j^k (1 - \hat{y}_j^k) (y_j^k - \hat{y}_j^k)$$

$$\Delta w_{hj} = \eta g_j b_h$$



$$\frac{\partial \sigma(y)}{\partial y} = \sigma(y)(1 - \sigma(y))$$

$$\frac{\partial E}{\partial w_i} = -\sum_{d \in D} \hat{y}_d (1 - \hat{y}_d) (y_d - \hat{y}_d) x_i$$

五、反向传播算法

类似的可以推导出：

$$\Delta\theta_j = -\eta g_j$$

$$\Delta v_{ih} = \eta e_h x_i$$

$$\Delta\gamma_h = -\eta e_h$$

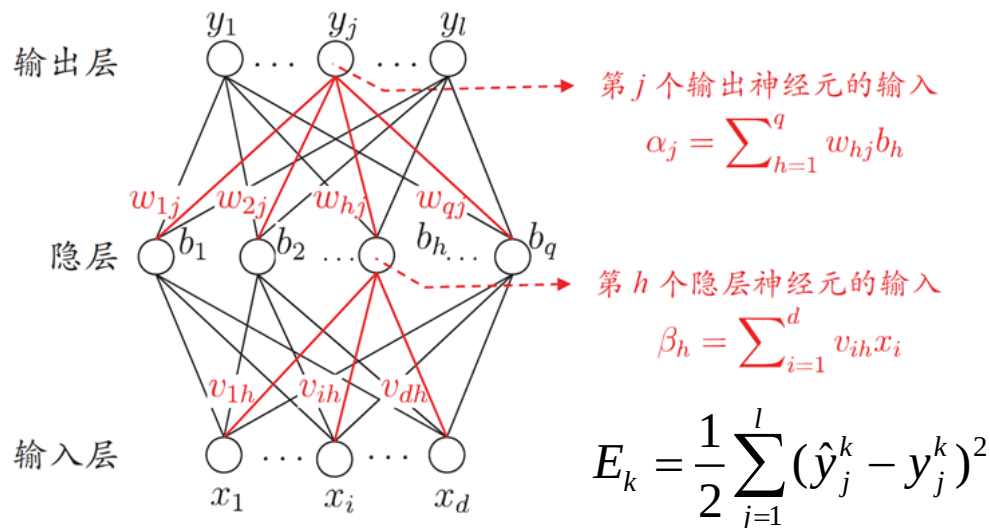
其中

$$e_h = \frac{\partial E_k}{\partial b_h} \cdot \frac{\partial b_h}{\partial \beta_h}$$

$$= \sum_{j=1}^l \frac{\partial E_k}{\partial \alpha_j} \cdot \frac{\partial \alpha_j}{\partial b_h} \sigma'(\beta_h - \gamma_h)$$

$$= \sum_{j=1}^l w_{hj} g_j \sigma'(\beta_h - \gamma_h)$$

$$= b_h(1-b_h) \sum_{j=1}^l w_{hj} g_j$$



- 学习率 η 控制着算法每一轮迭代中的更新步长, 若太长则让容易震荡, 太小则收敛速度又会过慢

五、反向传播算法(1)

输入：训练集 $D = \{(\mathbf{x}_k, \mathbf{y}_k)\}_{k=1}^m$;
学习率 η .

过程：

- 1: 在(0, 1)范围内随机初始化网络中所有连接权和阈值
- 2: **repeat**
- 3: **for all** $(\mathbf{x}_k, \mathbf{y}_k) \in D$ **do**
- 4: 根据当前参数和式(5.3) 计算当前样本的输出 $\hat{\mathbf{y}}_k$;
- 5: 根据式(5.10) 计算输出层神经元的梯度项 g_j ;
- 6: 根据式(5.15) 计算隐层神经元的梯度项 e_h ;
- 7: 根据式(5.11)-(5.14) 更新连接权 w_{hj}, v_{ih} 与阈值 θ_j, γ_h
- 8: **end for**
- 9: **until** 达到停止条件

输出：连接权与阈值确定的多层前馈神经网络

图 5.8 误差逆传播算法

五、反向传播算法(2)

BP算法:

1) 初始化神经网络的权和偏置。

2) 对实例中的每个训练样本:

隐层
输出

① 计算每个隐含层节点的输入: $I_j = \sum_i X_i W_{ij} - \theta_j$

② 计算每个隐含层节点的输出: $O_j = 1/(1 + e^{-I_j})$

输出层
输出

③ 计算每个输出层节点的输入: $I_k = \sum_j O_j W_{jk} - \theta_k$

④ 计算每个输出层节点的输出: $O_k = 1/(1 + e^{-I_k})$

误差
计算

⑤ 计算每个输出层节点的误差: $Err_k = O_k(1 - O_k)(T_k - O_k)$

⑥ 计算每个隐含层节点的误差: $Err_j = O_j(1 - O_j) \sum_k W_{jk} Err_k$

权值
更新

⑦ 更新输入层到隐含层的权值: $W_{ij} = W_{ij} + (l) X_i Err_j$

⑧ 更新隐含层到输出层的权值: $W_{jk} = W_{jk} + (l) O_j Err_k$

阈值
更新

⑨ 更新每个隐含层节点的偏置: $\theta_j = \theta_j - (l) Err_j$

⑩ 更新每个输出层节点的偏置: $\theta_k = \theta_k - (l) Err_k$

3) 判断终止条件是否满足, 若满足则停止, 否则重复执行步骤 2)。

基于梯度下降法则的
公式推导可参阅:
[周志华, 2016];
[Mitchell, 2003]。

五、反向传播算法（课本）

- 初始化每一个的网络权向量 w_i 为小的随机值(如-0.05~+0.05)
- 在满足终止条件之前，执行

➤ 循环每个训练样例 $\langle (x_1, \dots, x_m), t \rangle$ 执行

□ 把输入 $(x_1, \dots, x_m)$ 沿网络向前传播，并计算网络每一个单元 u 的输出 o_u
使误差沿网络反向传播

□ 对于网络每个输出单元 k ，计算误差项

$$\delta_k \leftarrow o_k(1-o_k)(t_k-o_k)$$

□ 对于网络的每个隐藏单元 h ，计算误差项

$$\delta_h \leftarrow o_h(1-o_h) \sum_{k \in \text{outputs}} w_{k,h} \delta_k$$

□ 更新每个网络权值 $w_{j,i}$

$$w_{j,i} \leftarrow w_{j,i} + \Delta w_{j,i} \quad \text{其中}$$

$$\Delta w_{j,i} = \eta \delta_j x_{j,i}$$

(注: $x_{j,i}$: 单元 i 的输出, 对应于 $w_{j,i}$)

$$w_i \leftarrow w_i + \eta (t - o) x_i \quad \frac{\partial \sigma(y)}{\partial y} = \sigma(y)(1 - \sigma(y)) \quad \frac{\partial E}{\partial w_i} = - \sum_{d \in D} o_d(1 - o_d)(t_d - o_d)x_i$$

误差
计算

权值
更新

五、反向传播算法

BP算法的基本过程：

- BP算法的基本工作过程大致可以分为三个阶段：
 - 1) **信号的前向传播阶段**：在这个阶段，要求计算出隐含层和输出层中每一神经元的网络净输入和网络输出。
 - 2) **误差的逆向传播阶段**：在这个阶段，要求计算出输出层和隐含层中每一神经元的误差。
 - 3) **权值和阈值的更新阶段**：在这个阶段，要求更新所有连接权的权值和所有M-P神经元的阈值。

五、反向传播算法

- 标准 BP 算法
$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}} \quad E_d(\vec{w}) = \frac{1}{2} \sum_{k \in \text{outputs}} (t_k - o_k)^2$$
 - 每次针对单个训练样例更新权值与阈值
 - 参数更新频繁, 不同样例可能抵消, 需要多次迭代
- 累计 BP 算法
$$E = \frac{1}{m} \sum_{k=1}^m E_k$$
 - 其优化的目标是最小化整个训练集上的累计误差
 - 读取整个训练集一遍才对参数进行更新, 参数更新频率较低
- 终止条件
 - 迭代次数达到固定值
 - 误差降到某个阈值以下
 - 分离的验证样例集合符合某个标准
 - 终止标准很重要: (太少) 误差大或 (太多) 过度拟合

五、反向传播算法

增加动量项 (Momentum)

- 反向传播算法的变体

- 使第n次迭代时的权值的更新部分地依赖与发生在第t-1次迭代时的更新

$$\Delta w_{hj}(t) = \eta g_h b_j + \alpha \Delta w_{hj}(t-1); \quad \Delta v_{ih}(t) = \eta e_h x_i + \alpha \Delta v_{ih}(t-1)$$

$0 \leq \alpha < 1$, $\Delta w_{hj}(t-1)$ 和 $\Delta v_{ih}(t-1)$ 被称为动量项

- 理解动量项: BP好似使球沿误差曲面的一个方向滚动
 - 有动量: 动量项有时会使球滚过误差曲面的局部极小值或使其滚过曲面上的平坦区域
 - 无动量: 在梯度不变的区域逐渐增大搜索步长的效果, 从而可以加快收敛

五、多层网络和反向传播算法

BP算法可能面临的问题：

1) 结构学习问题：

- ✓ 多层前馈神经网络包括：输入层、输出层、一个或多个隐含层。输入层神经元的个数由输入的数据维度（连续属性）和编码方法（离散属性）确定；输出层神经元的个数由待分类的类别数目和编码方法确定，比如4类问题的二进制编码需要2个神经元。
- ✓ [Hornik et al. , 1989]证明：只需要一个包含足够多神经元的隐含层，多层前馈神经网络就能以任意精度逼近任意复杂度的连续函数。
- ✓ 如何确定隐含层神经元的个数仍然是个悬而未决问题。实际应用中通常是根据经验来确定或者靠“试错法”来调整。

五、多层网络和反向传播算法

BP算法可能面临的问题：

2) 初始化问题：

- ✓ 在BP算法中，连接权和偏置在网络学习之前，都需要将其初始化为不同的小随机数。“不同”保证网络可以学习；“小随机数”可以防止其值过大而提前进入饱和状态，达到局部极小值。
- ✓ 解决办法：重新初始化。

五、多层网络和反向传播算法

BP算法可能面临的问题：

3) 步长设置问题：

- BP网络的收敛是基于无穷小的修改量，学习率控制着算法每一轮迭代中的更新步长。
 - 步长太小，收敛速度就会过慢
 - 步长太大，又可能会导致网络的不稳定、甚至瘫痪
 - 自适应步长，让步长随着网络的训练而不断变化

五、多层网络和反向传播算法

BP算法可能面临的问题：

4) 权值与阈值的更新问题：

- 基本的BP算法采用的是样例更新，即每处理一个训练样例就更新一次权值与阈值。
 - 样例更新的缺陷：参数更新频繁，不同样例可能抵消，需要迭代的次数较多。
 - 训练样例的输入顺序对训练结果有较大影响，它更“偏爱”较后输入的样例。
 - 给训练样例安排一个适当的顺序，又是非常困难的。
- 解决的办法就是采用周期更新，即每处理一遍所有的训练样例才更新一次权值与阈值。
 - 在很多实际任务中，周期更新的累计误差下降到一定程度后，进一步下降会非常缓慢，这时样例更新往往会获得较好的解，尤其当训练集非常大时效果更明显。

五、多层网络和反向传播算法

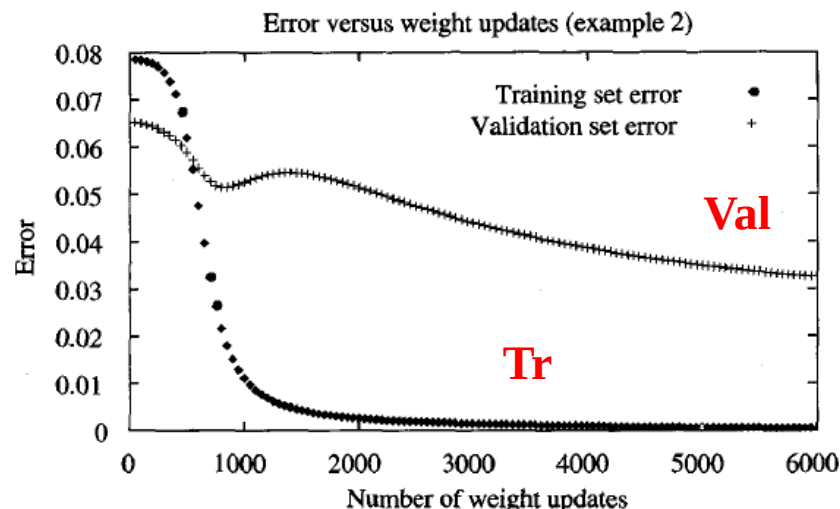
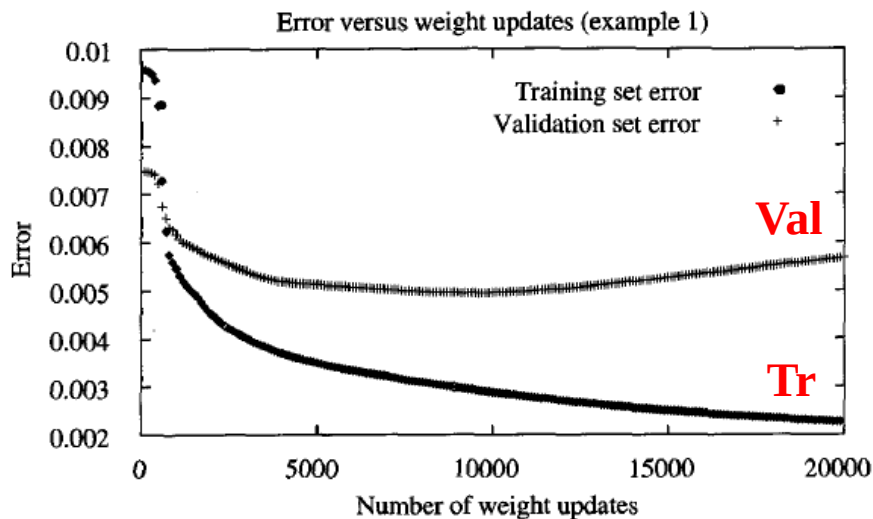
5) 过拟合问题

- 神经网络由于强大的表示能力，经常遭遇过拟合。具体表现为：训练误差持续降低，但测试误差却可能上升。
- 过拟合(泛化精度)
 - 权值拟合了训练样例的“特异性” (idiosyncrasy)
 - 大量权值参数为拟合这样的“特异性”提供了很大的自由度
- 为什么过度拟合往往发生在迭代的后期？
 - 权值被初始化为较小的随机值，只能描述光滑的表面
 - 随着训练的进行，权值开始增加，学习到的表面的复杂性也会增加
 - 权值调整迭代次数足够多，将会产生过于过度复杂的决策面，拟合了所有的样例

五、多层网络和反向传播算法

● 缓解过拟合的策略

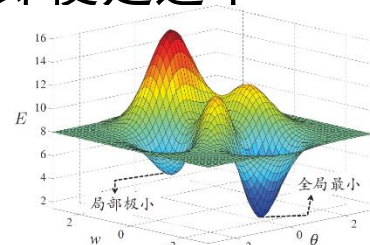
- ❑ **权值衰减**: 误差 E 小于阈值, 每次迭代以某个小因子降低每个权值
- ❑ **早停**就是在训练过程中, 若训练误差降低, 但验证误差升高, 则停止训练
- ❑ **正则化**就是在误差目标函数中增加一项描述网络复杂程度的部分, 比如连接权值与阈值的平方和
- ❑ 最成功的方法是交叉验证



五、多层网络和反向传播算法

收敛性和局部极小值

- BP算法仅保证收敛到误差 E 的某个局部极小值
 - 误差曲面可能含有多个不同的局部极小值
 - 梯度下降可能陷入这些局部极小值中的任何一个
- BP算法在实践中仍然是一种高效的函数逼近方法
 - 大量的网络权值为逃避局部最小值提供了“逃生路线”
 - 网络权值的演变方式
 - ✓ 在初始阶段，权值初始化接近于0，网络表现出一个平滑的函数(近似一个线性函数)
 - ✓ 随着权值的增长，它们会达到可以表示高度非线性网络函数的程度(权空间区域存在更多的局部极小值)
 - ✓ 当权达到这一点时它们已经足够靠近全局最小值，即便是这个区域的局部极小值也是可以接受的



五、多层网络和反向传播算法

缓解局部极小值的问题

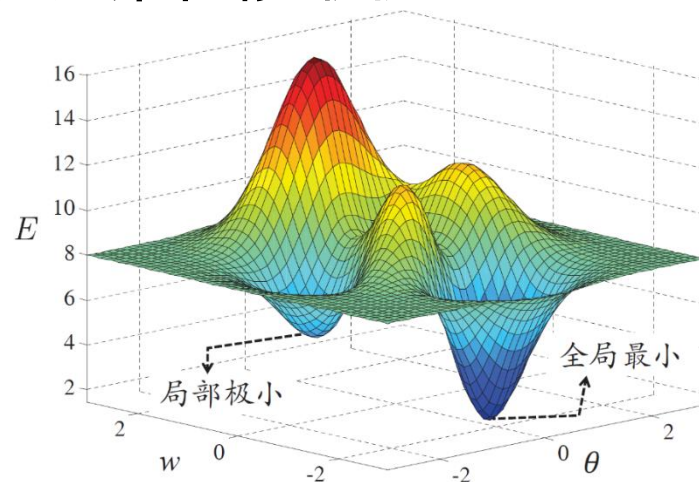
- 关于局部极小值问题的一些启发式规则：

- 添加动量项

- 使用随机梯度而不是真实梯度下降

- SGD对于每个训练样例沿一个不同的误差曲面有效下降，依赖于梯度的平均来近似对整个训练集合上的梯度。

- 训练多个网络，选择最佳网络（验证集性能最优）



五、多层网络和反向传播算法

前馈网络的表征能力

- 布尔函数：使用两层单元的网络
 - 表示任意布尔函数的通用方案
 - 对于每个输入创建不同的隐单元，设置其权值使仅在特定的向量输入到网络时，这个单元被激活，即输出单元实现为一个仅由所希望输入模式激活的或门（OR gate）
- 有界的连续函数 (Cybenko 1989, Hornik 1989)
 - 两层单元网络：隐藏层sigmoid单元+ 输出层线性单元网络
 - 任意小的误差逼近
- 任意函数 (Cybenko 1988)
 - 三层单元的网络(两层sigmoid单元+ 一个线性输出层)
 - 任意精度逼近

注意：梯度下降是从一个初始的权值开始的，因此搜索范围内的网络权向量可能不包含所有的权向量

五、多层网络和反向传播算法

假设空间搜索和归纳偏置

- 反向传播算法的假设空间

- n 维欧氏空间、连续空间
- E 是可微的
 - 梯度下降算法可以搜索到一个获得最佳假设

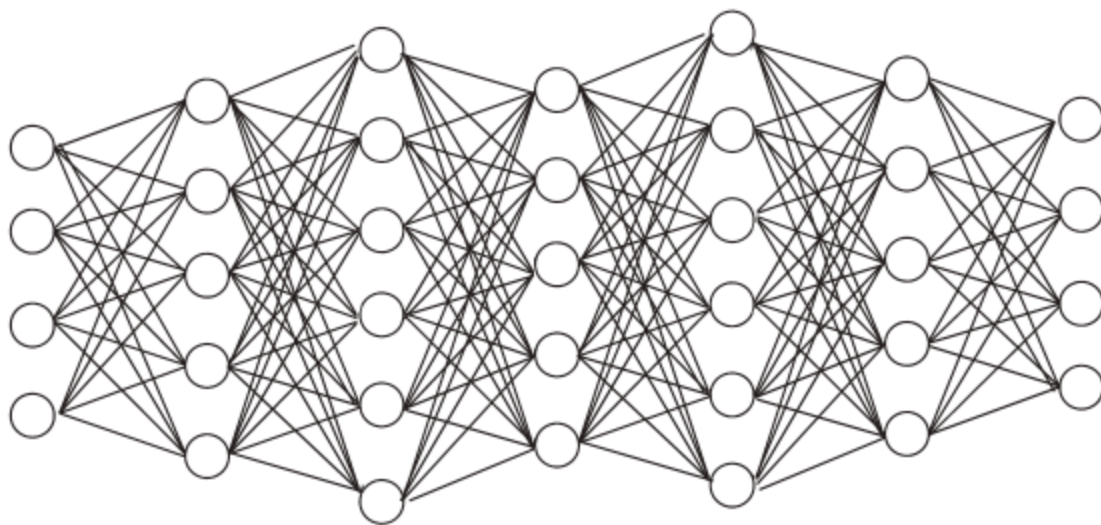
- 反向传播算法的归纳偏置

- 依赖于
 - 梯度下降搜索
 - 权空间覆盖可表征函数空间的方式的相互作用性
- 粗略地刻画为在数据点之间平滑插值
 - 给定俩正，无反例，BP倾向于把这两点之间的点也标记为正例（平滑变化的决策区域）

六、深层神经网络

深层神经网络的定义

- 随着云计算、大数据时代的到来，计算能力的大幅提高可缓解训练低效性，训练数据的大幅增加则可降低过拟合风险，因此，以深度学习(deep learning)为代表的复杂模型开始受到人们的关注。
- 典型的深度学习模型就是很深层的神经网络：包含2个以上隐含层的神经网络。



六、深层神经网络

深层神经网络的定义

- 模型复杂度

- 增加隐含层神经元的数目（模型宽度）
- 增加隐含层的数目（模型深度）
- 从增加模型复杂度的角度来看，增加隐含层的数目比增加隐含层神经元的数目更有效。这是因为增加隐含层的数目不仅增加了拥有激活函数的神经元数目，还增加了激活函数嵌套的层数。

- 深层模型的难点问题

- 多隐含层网络难以直接用经典算法（例如标准的BP算法）进行训练，因为误差在多隐含层内逆传播时，往往会发散，而不能收敛到稳定状态。

六、深层神经网络

深层神经网络的学习

● 预训练+微调

- 预训练：无监督逐层训练是多隐含层网络训练的有效训练手段，**每次训练一层隐含层结点**，训练时将上一层隐含层结点的输出作为输入，而本层隐含层结点的输出作为下一层隐含层结点的输入，这称为“**预训练**”。
- 微调：预训练全部完成后，再对整个网络进行微调训练。微调一般使用BP算法。
- 例子：深度信念网络(DBN)[Hinton et al.,2006]。
 - 结构：每一层都是一个受限Boltzmann机
 - 训练：无监督预训练+BP算法微调
 - 分析：**预训练+微调**的做法可以视为将大量参数进行分组，对每组**先找到局部**看起来比较好的设置，然后再基于这些局部较优的结果联合起来进行**全局寻优**。

六、深层神经网络

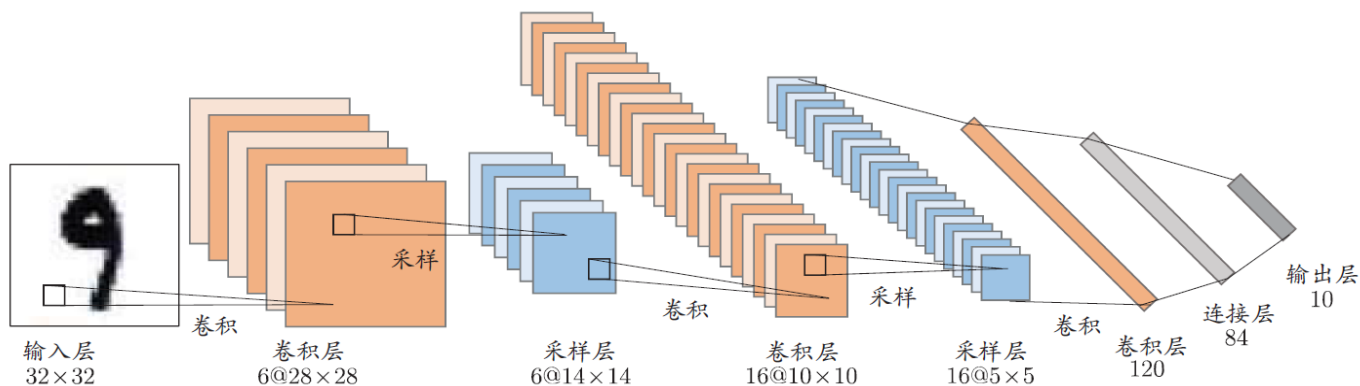
深层神经网络的学习

- 权共享

- 一组神经元共享相同的连接权值。
- 例子：卷积神经网络(CNN) [LeCun and Bengio, 1995; LeCun et al., 1998]。
 - 结构：CNN复合多个卷积层和采样层对输入信号进行加工，然后在连接层实现与输出目标之间的映射。
 - 训练：BP算法。
 - 分析：在训练中，无论是卷积层还是采样层，其每一组神经元（即图下中的每个平面）都共享相同的连接权，从而大幅减少了需要训练的参数数目；另外，在CNN中通常将Sigmoid 激活函数替换为修正的线性函数 $f(x)=\max(0,x)$ 。

六、深层神经网络

深层神经网络的学习



卷积神经网络用于
手写数字识别
[LeCun et al., 1998]

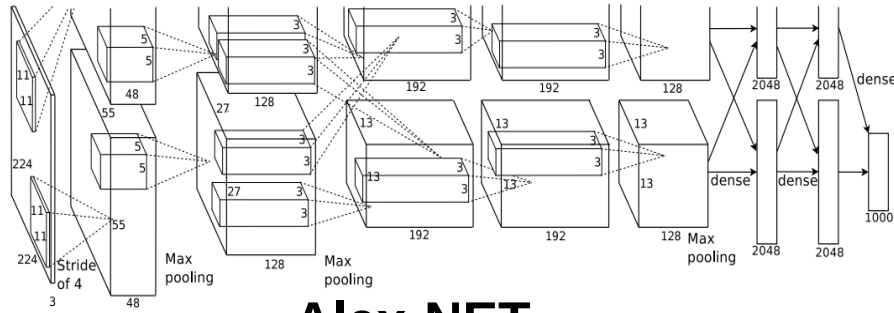
- ✓ **卷积层：**每个卷积层包含多个特征映射，每个特征映射是一个由多个神经元构成的“平面”，通过一种**卷积滤波器**提取输入的一种特征。
- ✓ **采样层：**也叫“汇合层”，其作用是**基于局部相关性原理进行亚采样**，从而在减少数据量的同时保留有用信息。
- ✓ **连接层：**每个神经元被全互连接到上一层每个神经元，本质就是传统的神经网络，其目的是通过连接层和输出层的连接完成识别任务。

六、深层神经网络

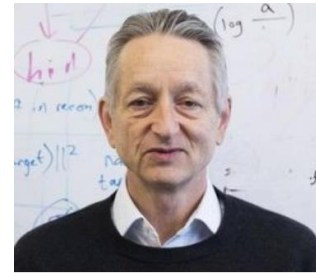
深度学习的理解

- 深度学习通过多层处理，逐渐将初始的低层特征表示转化为高层特征表示后，用“简单模型”即可完成复杂的分类学习等任务。由此可将深度学习理解为进行“特征学习”(feature learning) 或 “表示学习”(representation learning)。
- 非深度学习技术用于解决现实任务时，描述样本的特征通常需由人类专家来手工设计，这称为“特征工程”(feature engineering)。众所周知，特征的好坏对模型的泛化性能有至关重要的影响，要人类专家手工设计出好特征并非易事；而特征学习则通过深度学习技术自动产生好特征，这使得机器学习向“全自动数据分析”又前进了一步。

六、深层卷积神经网络家族



Alex-Net



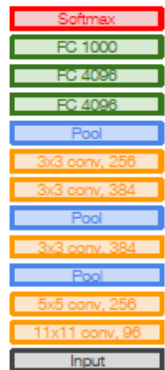
Geoffrey Hinton



Yann LeCun



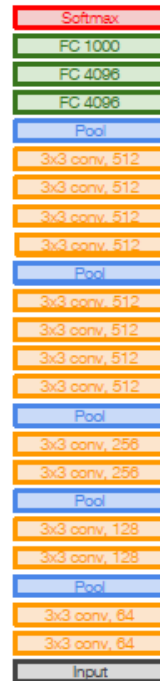
Yoshua Bengio



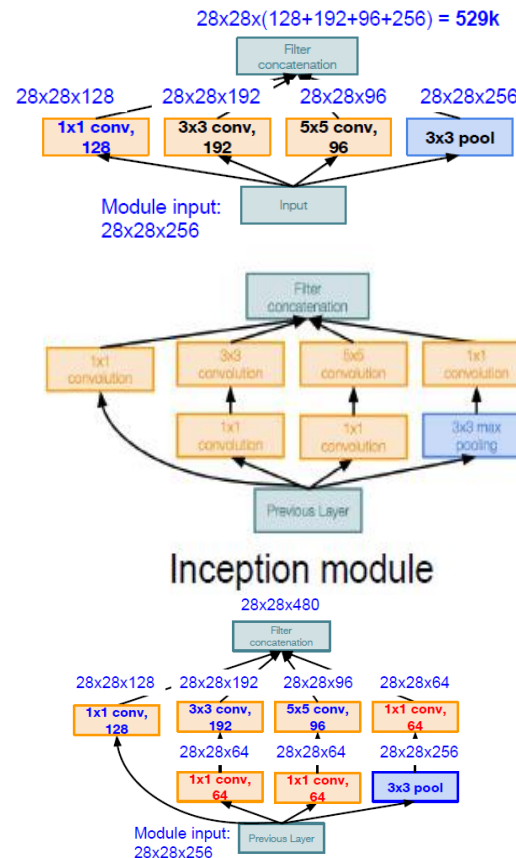
AlexNet



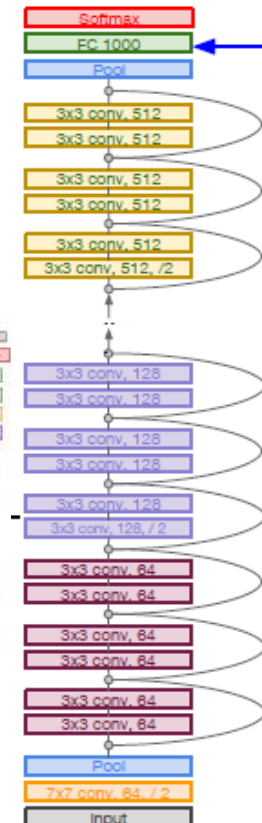
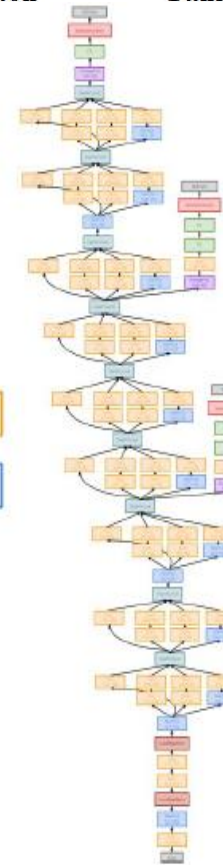
VGG16



VGG19



Inception module



No FC layers besides FC 1000 to output classes

Global average pooling layer after last conv layer

七、小结和补充读物

- 感知器、感知器训练法则及表示能力
- Delta法则：GD、SGD
- 多层神经网络学习为学习实数值和向量值函数提供了一种实际的方法，适用于连续的和离散值的属性学习
- 对训练数据中的噪声具有很好的健壮性
- 反向传播算法考虑的假设空间是固定连接的有权网络所能表示的所有函数的空间
- 反向传播算法使用梯度下降搜索可能的假设空间，以迭代减少网络中的误差以拟合训练数据
- 反向传播算法收敛到局部极小值
- BP可以创造出网络输入中未明确出现的特征
- 多层网络有较强的表征能力：布尔，有界连续，任意
- 过度拟合及如何避免（WD，早停，正则化，交叉验证）
- 作业：4.2, 4.3, 4.5, 4.7

举例：人脸识别

- 训练数据

- 20个不同人的摄影图像
- 每人32张
 - 不同表情
 - ✓ 快乐、伤心、愤怒、中性
 - 面部方向
 - ✓ 左、右、正前、上
 - 其他
 - ✓ 是否带太阳镜，人后面的背景，穿着
- 共收集624灰度图像
 - 分辨率 120x128
 - 色素： 0 --- 255

- 任务：学习图像中人脸的朝向

举例：人脸识别

设计要素

- 输入编码

- 表示形式1：提取边缘，亮度一致的区域或其他局部图像特征
 - ✓ 问题：导致每幅图像有不同数量的特征参数，而ANN具有固定数量的输入单元
- 表示形式2：每个像素一个网络输入
 - ✓ 固定输入数量
 - ✓ 像素强度值线性缩放到0到1的范围内（以便网络输入的值与隐藏单元和输出单元激活的间隔相同）

- 输出编码

- 使用单一的输出单元
- 使用四个不同的输出单元
 - ✓ 取具有最高值的输出作为网络的预测值，这种方法被称为n取1（1-of-n）输出编码

举例：人脸识别

设计要素

- 为什么选择1-of-n输出编码
 - 提供更大的自由度来表示目标函数
 - 提供网络预测的置信度
- 4个输出单元的目标值
 - 一个显而易见的选择 $\langle 1, 0, 0, 0 \rangle \dots$
 - 使用0.1 and 0.9, $\langle 0.9, 0.1, 0.1, 0.1 \rangle \dots$
 - 避免使用0和1的原因
 - Sigmoid 单元不能产生这样的输出
 - 避免权值无限增长
 - Sigmoid 单元在有限权值情况下可实现0.1和0.9的值

举例：人脸识别

设计要素

● 网络结构

- 网络包含多少个单元以及如何互连
 - 分层网络，一层的每个单元向前连接到下一层的每个单元
 - ✓ 两层sigmoid单元
 - 隐藏单元的数量
 - ✓ 3个，90%的测试精度，Sun SParc5工作站上训练5分钟，260 幅训练图像
 - ✓ 30个，提高一到两个百分比，训练接近一小时
- 隐藏单元数量的启示
 - 需要最少数量的隐藏单元
 - 多余的隐藏单元不会显著提高泛化精度
 - 过度拟合：增加隐藏单元的数量通常会增加过度拟合训练数据的倾向

举例：人脸识别

设计要素

- 学习算法的其他参数

- 学习速率:0.3, 冲量 : 0.3
 - ✓ 较低的值产生大体相当的泛化精度, 但训练时间较长
 - ✓ 这两个值被设定太高, 训练将不能收敛
- 使用完全的梯度下降
- 输出单位中的权值初始化为较小的随机值
- 输入单位权值初始化为零
- 训练迭代次数:
 - ✓ 由训练集和独立的验证集决定
- 最终网络:
 - ✓ 对验证集合精度最高的网络
- 最终报告精度
 - ✓ 在第三个集合—测试集合上测量得到

举例：人脸识别

学习到的隐藏层表示

- 1次迭代后输出权值
 - 1次迭代后，从输入单元到隐藏单元的权值
- 100次迭代后输出权值
 - 100次迭代后，从输入单元到隐藏单元的权值



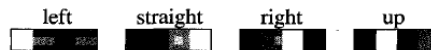
30 × 32 resolution input images



Network weights after 1 iteration through each training example



30 × 32 resolution input images



Network weights after 100 iterations through each training example